AWS ARCHITECT ASSOCIATE | PROJECT 01

AWS VPC Peering: Secure Communication

Between Two Virtual Private Clouds

Project Report

Field	Details
Student name	Yashika
Batch	AWS Internship Batch 1
Institution	Indira Gandhi Delhi Technical University for Women (IGDTUW)
Department	Computer Science and Engineering
Course / subject	AWS Architect Associate (EWB Courses): AWS Networking, Project 01: VPC Peering
AWS Region	Asia Pacific (Sydney), ap-southeast-2
Completion / submission date	26 October 2026

Abstract

This project demonstrates private network communication between two Amazon Virtual Private Clouds (VPCs) in the Asia Pacific (Sydney) Region using VPC peering. VPC A (10.10.0.0/16) contains a public bastion host, ewb-bastion (10.10.1.211), and a private application instance, App-A (10.10.2.59). VPC B (10.20.0.0/16) contains a private application instance, App-B (10.20.1.109). Neither application instance has a public IPv4 address, and VPC B has no Internet Gateway. The peering connection pcx-095011188f7b136a1 (ewb-a-b-peer) is Active, and routes were configured in three custom route tables.

Captured test evidence shows that the bastion reached both application instances with ICMP (4 of 4 replies in each case), and that App-A reached App-B with ICMP (4 of 4 replies, 0% loss). SSH from the laptop reached App-A and App-B through the bastion using ProxyCommand. A TCP connection to port 22 from App-B to App-A failed twice and then succeeded after a security-group rule was added. A TCP port 22 test from App-A to App-B also returned a failed attempt followed by a successful one. An SSH attempt from App-A to App-B reached the remote service but was refused at public-key authentication. All results in this report are taken from the captured screenshots.

Index

Table 1: Contents of this report

No.	Section	Page
1	Abstract	2
2	Introduction	4
3	Objectives	4
4	Requirements	4
5	Architecture Design	5
6	Implementation	6
7	Configuration Details	11
8	Connectivity Testing	16
9	Troubleshooting	19
10	Results and Discussion	20
11	Security Considerations	20
12	Limitations and Future Improvements	21
13	Conclusion	22
14	References	22
15	Appendix A. Figure Index	23
16	Appendix B. Evidence Coverage	24

1. Introduction

1.1 Amazon Virtual Private Cloud

Amazon Virtual Private Cloud (VPC) allows a customer to define a logically isolated virtual network within an AWS Region. The customer selects an IPv4 address range (CIDR block), divides it into subnets, controls routing with route tables, and filters traffic to individual resources with security groups. Resources such as EC2 instances are launched into subnets and receive private IP addresses from the subnet range.

1.2 VPC Peering

A VPC peering connection is a networking link between two VPCs that allows resources in either VPC to communicate over private IP addresses. Traffic stays on the AWS network and does not traverse the public internet. A peering connection does not move traffic by itself. Each VPC must also have route table entries that direct traffic for the other VPC's CIDR to the peering connection, and the destination security rules must permit the traffic.

1.3 Purpose of This Project

Many cloud workloads are placed in separate networks for isolation, ownership, or lifecycle reasons, yet they still need to exchange data privately. This project implements that pattern at small scale and uses it to practise the relationship between routing, security rules, and the bastion-host administration model.

2. Objectives

- Plan non-overlapping IPv4 CIDR blocks for two VPCs and their subnets.
- Create public and private subnets, with an Internet Gateway attached to VPC A only.
- Launch a bastion host in the public subnet and two application instances in private subnets.
- Request, accept, and verify an active VPC peering connection between VPC A and VPC B.
- Configure forward and return routes in the relevant route tables.
- Restrict inbound security-group rules to individual lab hosts.
- Reach both private instances through the bastion host using SSH ProxyCommand, without copying the private key onto any instance.
- Test connectivity, record the results accurately, and troubleshoot any failures.

3. Requirements

The table lists the account, compute, and client requirements for the lab.

Table 2: Project requirements and environment

Requirement	Value used in this project
Cloud platform	Amazon Web Services, personal AWS account
Region	Asia Pacific (Sydney), ap-southeast-2
Networking service	Amazon VPC, including VPC peering
Compute service	Amazon EC2: three instances (one bastion, two application hosts)
Operating system	Amazon Linux 2023
Instance type	t3.micro
SSH key pair	ewb-lab (RSA, .pem format); private key stored on the local computer only
Client	Windows computer with OpenSSH client; ProxyCommand used for the jump
Existing default VPC	Present (172.31.0.0/16); preserved and not used by this project

4. Architecture Design

The design uses two VPCs with non-overlapping address ranges. VPC A is split into a public subnet, which hosts the bastion, and a private subnet, which hosts App-A. VPC B has a single private subnet for App-B. The two VPCs are connected by one peering connection. Figure 1 shows the design.

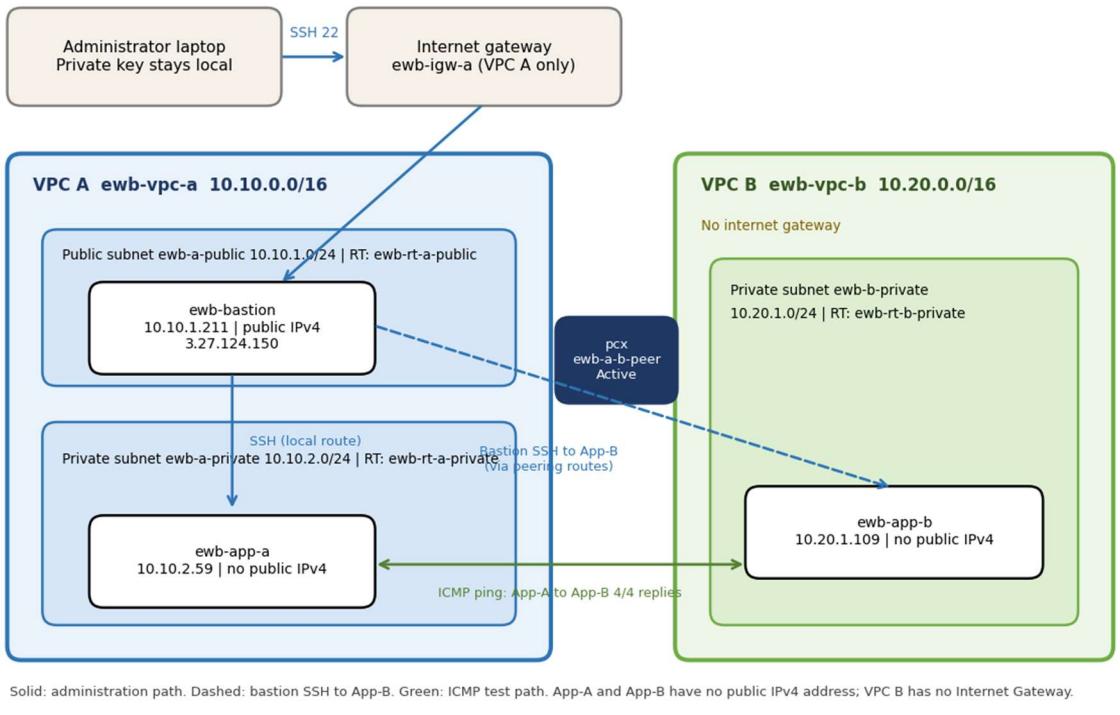


Figure 1: Network architecture of the project. Logical diagram drawn for this report, not a console screenshot.

4.1 Address Plan

Table 3: Address plan

Resource	Name	ID	CIDR / address
VPC A	ewb-vpc-a	vpc-03f34e01b8812f7fd	10.10.0.0/16
Public subnet (VPC A)	ewb-a-public	subnet-07d2cbbe162d5b452	10.10.1.0/24
Private subnet (VPC A)	ewb-a-private	subnet-08f85a1317d29abee	10.10.2.0/24
VPC B	ewb-vpc-b	vpc-00d4dd35acf4bda5f	10.20.0.0/16
Private subnet (VPC B)	ewb-b-private	subnet-08dd5098670da1e48	10.20.1.0/24
Bastion (VPC A, public)	ewb-bastion	i-09761540914901ad1	10.10.1.211, public 3.27.124.150
App-A (VPC A, private)	ewb-app-a	i-0d8d858fa0440bd30	10.10.2.59, no public IPv4
App-B (VPC B, private)	ewb-app-b	i-0f9cf32d123e7d889	10.20.1.109, no public IPv4

4.2 Design Decisions

Non-overlapping CIDRs. Peering requires that the two VPC ranges do not overlap. 10.10.0.0/16 and 10.20.0.0/16 satisfy this requirement.

Public and private subnets. Only the bastion sits in a subnet with an Internet Gateway route, and only the bastion has a public IPv4 address. App-A and App-B have no route to the internet and no public address.

Single entry point. All administrative access enters through the bastion, which limits the exposed attack surface to one host.

No Internet Gateway in VPC B. Peering does not provide edge-to-edge routing, so VPC B cannot use VPC A's Internet Gateway.

5. Implementation

The implementation followed the sequence below. Evidence for each stage is referenced in the final column.

Table 4: Implementation sequence

Step	Action taken	Result	Evidence
1	Created VPC A (ewb-vpc-a, 10.10.0.0/16) and VPC B (ewb-vpc-b, 10.20.0.0/16), both as VPC only with default tenancy	Both VPCs available	Figure 3; Figure 4
2	Created subnets ewb-a-public (10.10.1.0/24), ewb-a-private (10.10.2.0/24), ewb-b-private (10.20.1.0/24)	Subnets available with planned CIDRs	Subnet IDs in Section 4.1
3	Created ewb-igw-a and attached it to VPC A only	Gateway attached; VPC B has none	Figure 6

Step	Action taken	Result	Evidence
4	Created route tables ewb-rt-a-public, ewb-rt-a-private, ewb-rt-b-private and associated each with its subnet	Three custom route tables	Figure 9 to Figure 11
5	Launched the bastion, App-A and App-B (Amazon Linux 2023, t3.micro) with planned private IPs	Instances running; App-A and App-B without public IPv4	Figure 5
6	Created ewb-sg-bastion, ewb-sg-app-a and ewb-sg-app-b and added inbound rules	Rules restricted to individual hosts; see Section 6.5	Figure 12 to Figure 14
7	Created key pair ewb-lab; stored the .pem on the laptop; connected through the bastion	SSH to bastion, App-A and App-B	Figure 7; Figure 8; Figure 20
8	Requested peering ewb-a-b-peer (VPC A to VPC B), accepted it, and confirmed Active status	pcx-095011188f7b136a1 Active	Figure 2
9	Added peer routes: 10.20.0.0/16 to pcx in the VPC A tables; 10.10.0.0/16 to pcx in ewb-rt-b-private	Forward and return routes in place	Figure 10; Figure 11
10	Ran connectivity tests (Section 7) and a rule change on ewb-sg-app-a for the reverse TCP test	Results in Section 7	Figure 16 to Figure 21

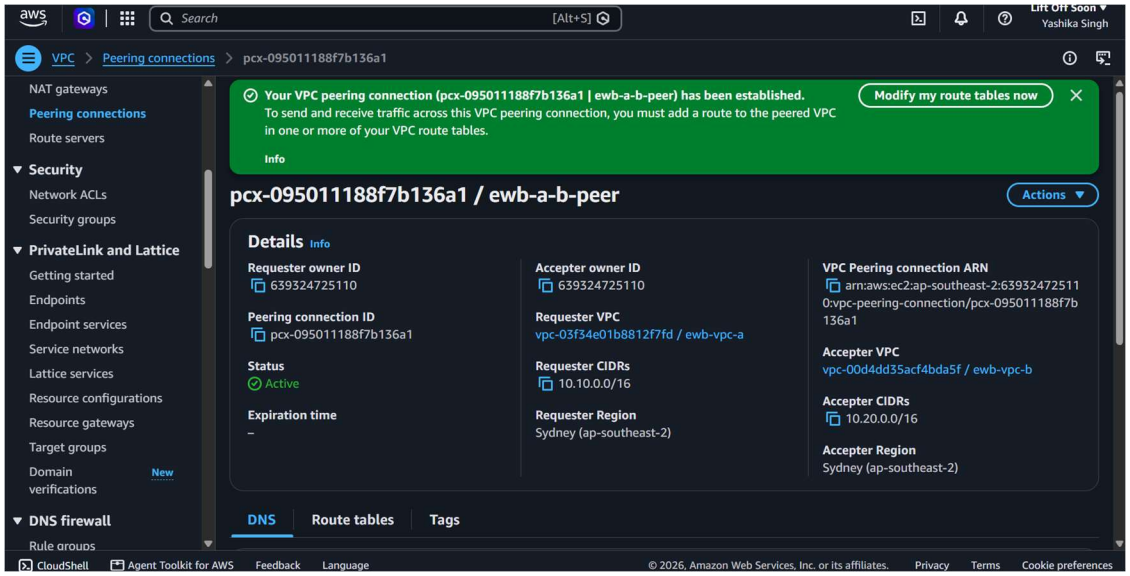


Figure 2: Peering connection ewb-a-b-peer (pcx-095011188f7b136a1) in Active status. Requester ewb-vpc-a (10.10.0.0/16), accepter ewb-vpc-b (10.20.0.0/16).

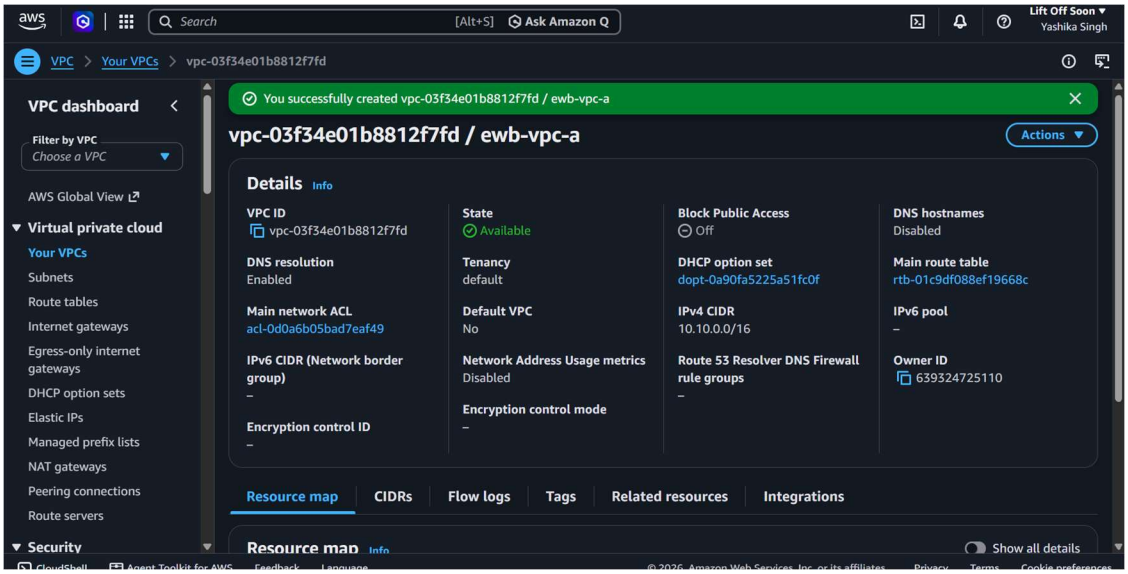


Figure 3: VPC A, ewb-vpc-a (vpc-03f34e01b8812f7fd), created as VPC only with IPv4 CIDR 10.10.0.0/16.

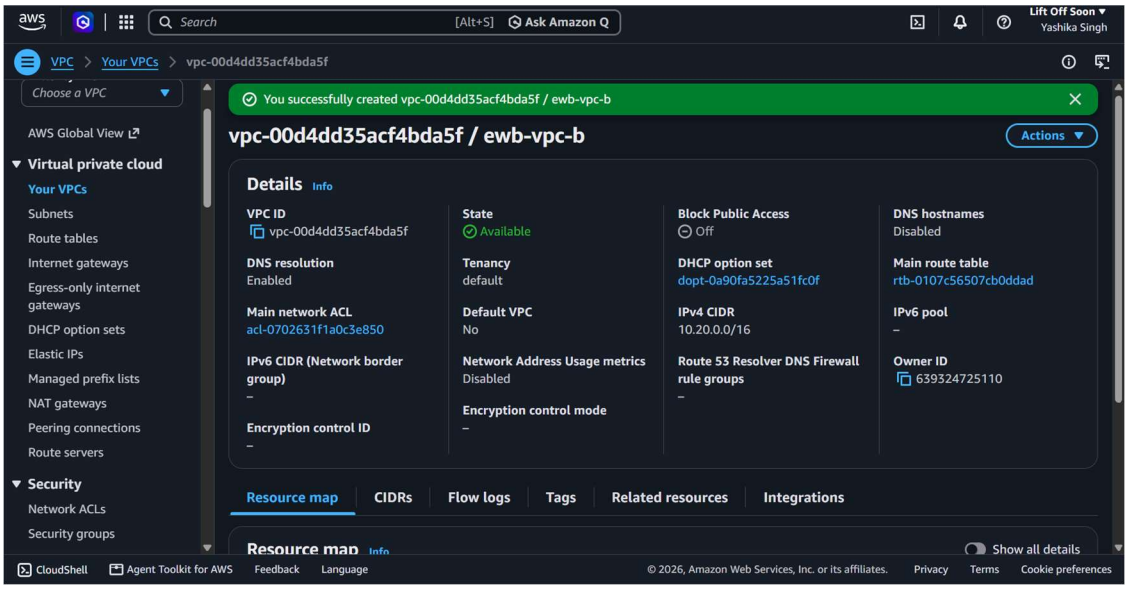


Figure 4: VPC B, ewb-vpc-b (vpc-00d4dd35acf4bda5f), IPv4 CIDR 10.20.0.0/16, with no Internet Gateway attached.

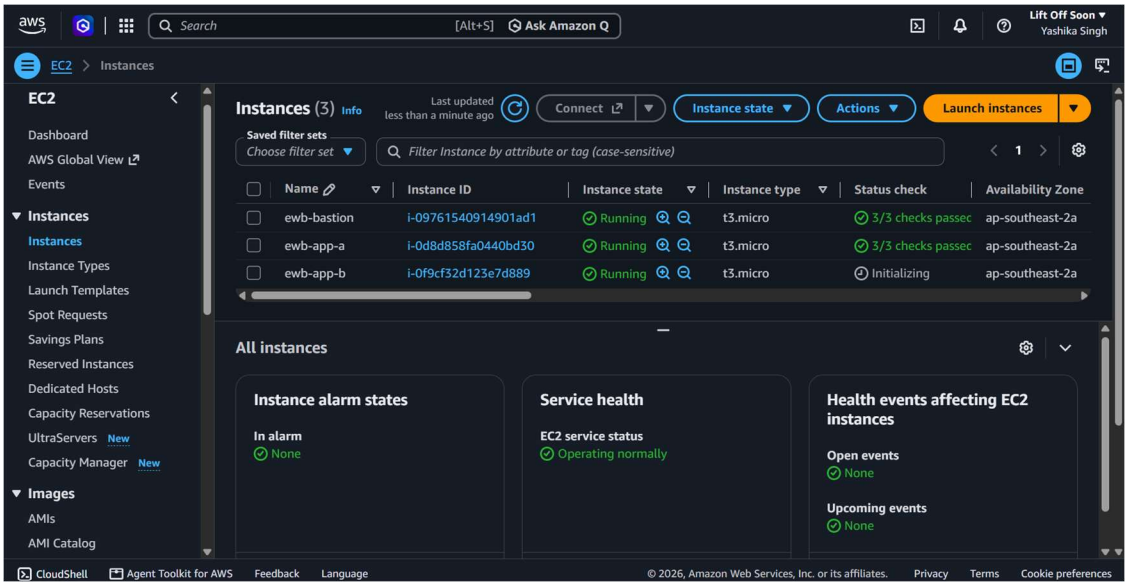


Figure 5: EC2 instances ewb-bastion, ewb-app-a and ewb-app-b on t3.micro. The bastion and App-A show 3/3 status checks passed; App-B was still initialising when captured.

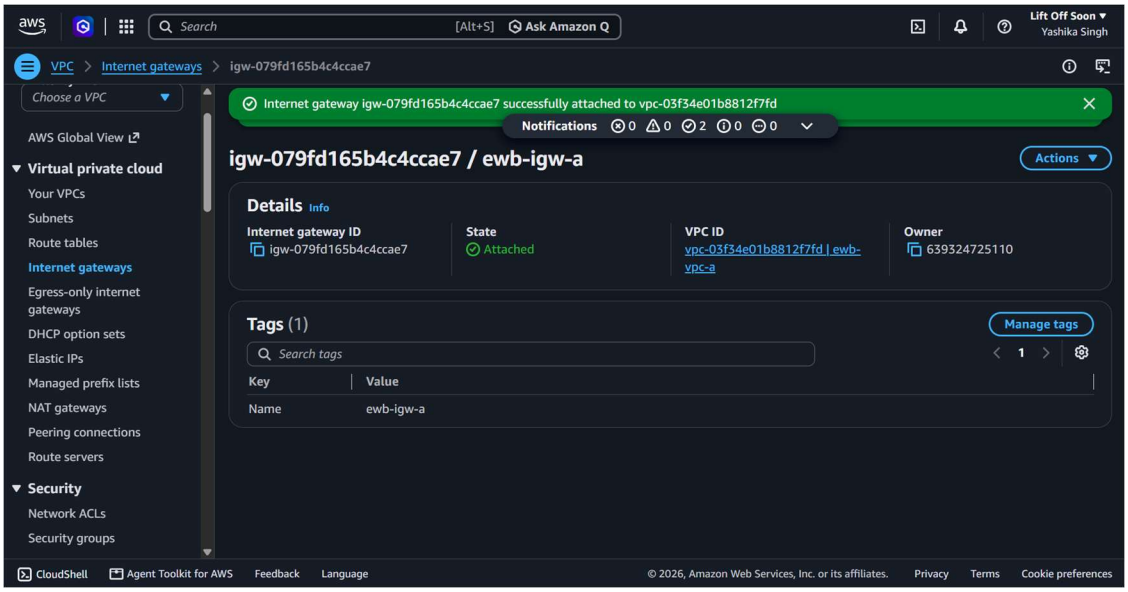


Figure 6: Internet gateway igw-079fd165b4c4ccae7 (ewb-igw-a) attached to VPC A (vpc-03f34e01b8812f7fd).

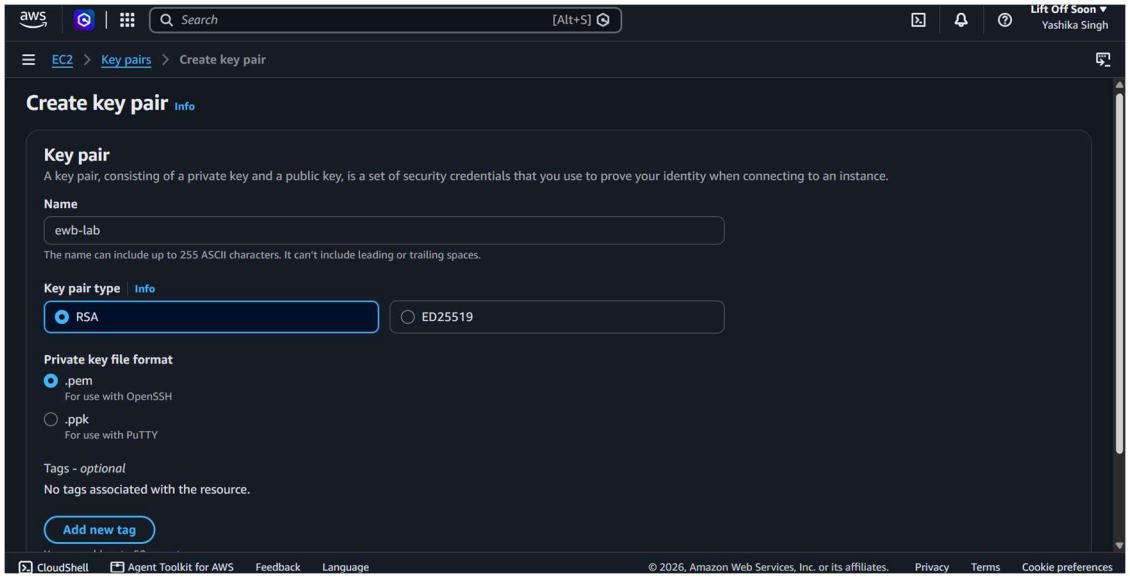


Figure 7: Creation of the RSA key pair ewb-lab, downloaded in .pem format and kept on the local computer.

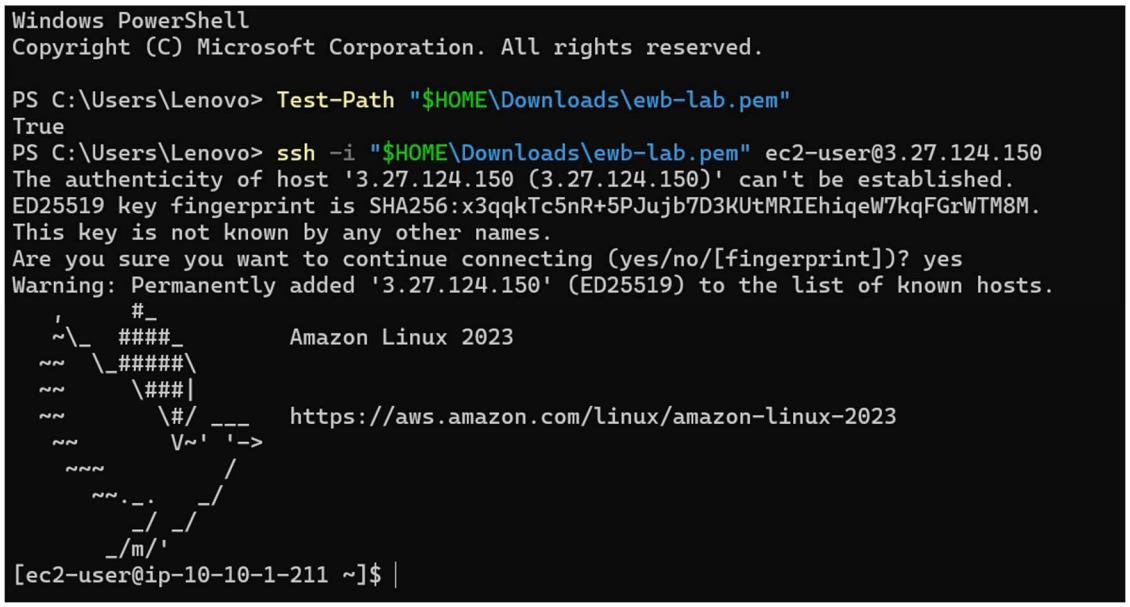


Figure 8: First SSH session from the laptop to ewb-bastion (3.27.124.150). The shell reports hostname ip-10-10-1-211, the bastion's private address.

5.1 Connecting Through the Bastion

The private instances have no public IP addresses, so they were reached through the bastion using SSH ProxyCommand. The laptop opened an SSH session to the bastion, which forwarded a TCP connection to the private target on port 22. The SSH handshake and key authentication then ran end-to-end between the laptop and the target. The private key was not copied to any EC2 instance. The command form used was:

ssh -i "$HOME\Downloads\ewb-lab.pem" -o "ProxyCommand=ssh -i $HOME\Downloads\ewb-lab.pem -W %h:%p ec2-user@3.27.124.150" ec2-user@10.10.2.59

The same pattern was used for App-B at 10.20.1.109. The bastion's public IP (3.27.124.150) is auto-assigned and will change if the instance is stopped and started, so the SSH command and the operator's security-group rule would then need updating.

6. Configuration Details

6.1 Resource Inventory

Table 5: Resource inventory

Name	Instance ID	VPC / subnet	Private IP	Public IPv4	Security group
ewb-bastion	i-09761540914901ad1	VPC A / ewb-a-public	10.10.1.211	3.27.124.150	ewb-sg-bastion
ewb-app-a	i-0d8d858fa0440bd30	VPC A / ewb-a-private	10.10.2.59	None	ewb-sg-app-a
ewb-app-b	i-0f9cf32d123e7d889	VPC B / ewb-b-private	10.20.1.109	None	ewb-sg-app-b

The instances were captured as running (Figure 5). App-B was still initialising at the time of that capture.

6.2 Route Tables

Table 6: Route tables as configured

Route table (ID)	Destination	Target	Purpose
ewb-rt-a-public (rtb-069a40c949c665db4)	10.10.0.0/16	local	Bastion to VPC A subnets
ewb-rt-a-public	0.0.0.0/0	igw-079fd165b4c4ccae7 (ewb-igw-a)	Internet access for the bastion
ewb-rt-a-public	10.20.0.0/16	pcx-095011188f7b136a1	Bastion SSH to App-B
ewb-rt-a-private (rtb-0f5d0567dfce2bee9)	10.10.0.0/16	local	App-A to VPC A subnets
ewb-rt-a-private	10.20.0.0/16	pcx-095011188f7b136a1	Forward route: App-A to App-B
ewb-rt-b-private (rtb-0e05059307348f9db)	10.20.0.0/16	local	App-B to VPC B subnets
ewb-rt-b-private	10.10.0.0/16	pcx-095011188f7b136a1	Return route: App-B to VPC A

Figures 9, 10 and 11 show the route tables in the console. The ewb-rt-a-private table has no default route, so App-A has no internet path. The ewb-rt-b-private table has no Internet Gateway route, consistent with VPC B having no Internet Gateway.

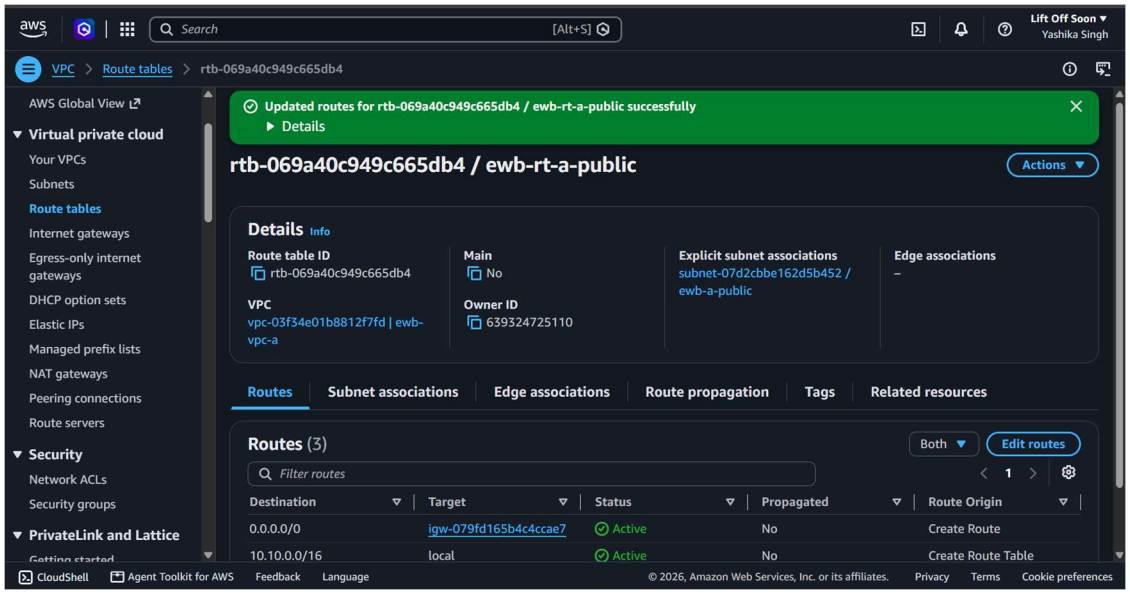


Figure 9: ewb-rt-a-public (rtb-069a40c949c665db4) after the route update: 0.0.0.0/0 to the Internet Gateway, 10.10.0.0/16 local, and a third route for the peer CIDR. Explicitly associated with ewb-a-public.

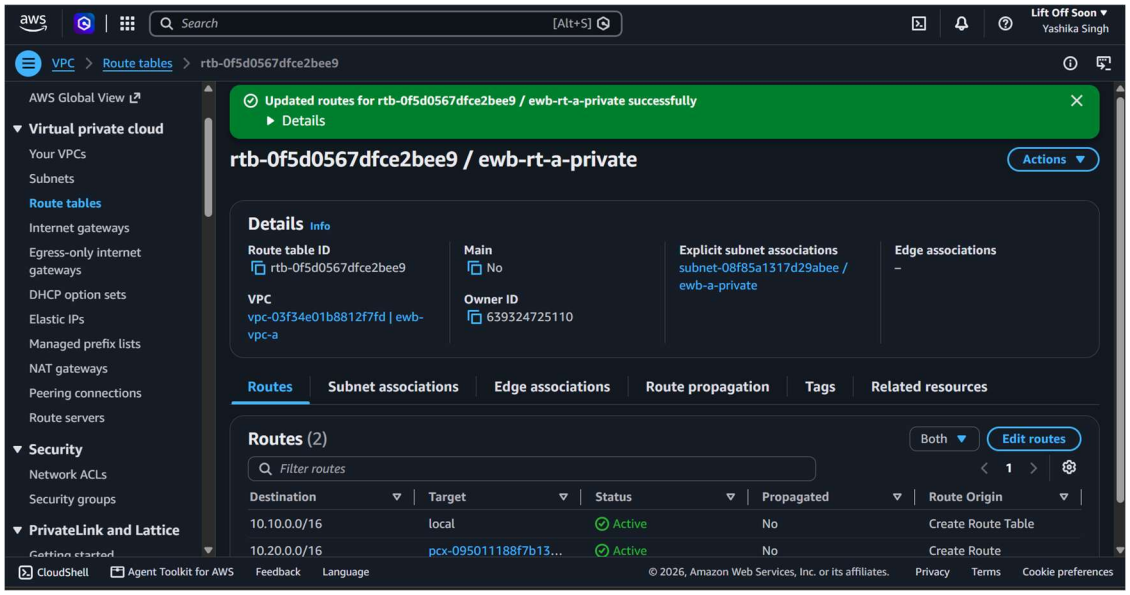


Figure 10: ewb-rt-a-private (rtb-0f5d0567dfce2bee9): local route 10.10.0.0/16 and 10.20.0.0/16 to pcx-095011188f7b136a1, both Active. No default route.

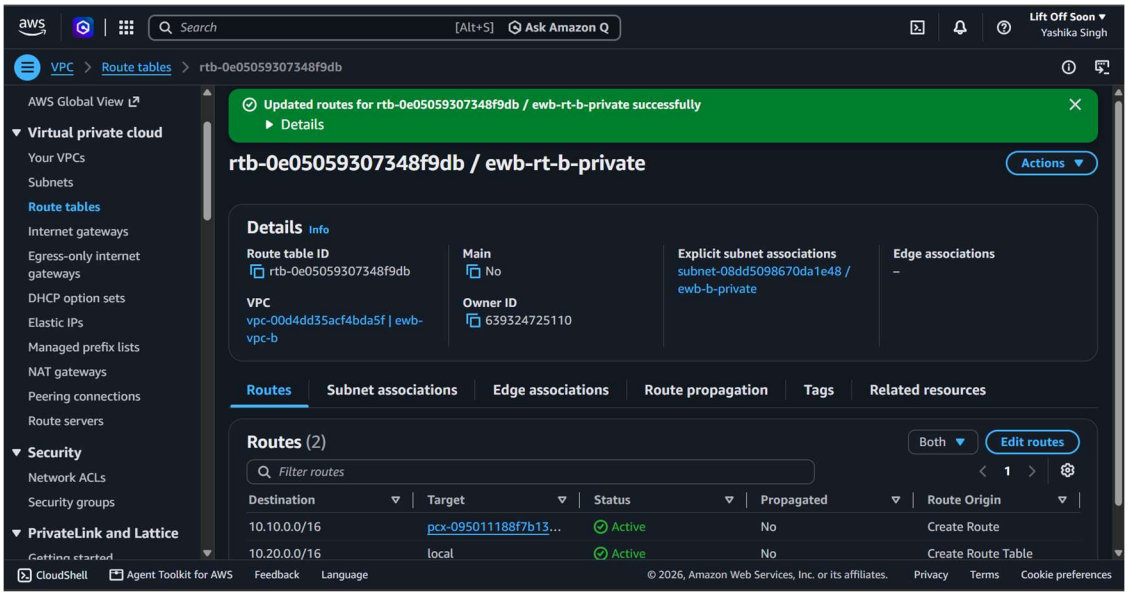


Figure 11: ewb-rt-b-private (rtb-0e05059307348f9db): return route 10.10.0.0/16 to pcx-095011188f7b136a1 and the local route, both Active.

6.3 Security Groups

The table lists the inbound rules in the form used for the captured tests. Section 6.5 explains the address correction that was applied.

Table 7: Security groups as recorded

Security group (ID)	Type / port	Source	Purpose
ewb-sg-bastion (sg-04f82b460e4b92f6a)	SSH / TCP 22	Operator public IPv4 /32	Only the administrator's address can reach the bastion
ewb-sg-app-a (sg-082eca4e694163d42)	SSH / TCP 22	Bastion private IP 10.10.1.211/32	Administrative SSH from the bastion
ewb-sg-app-a	All ICMP, IPv4	App-B private IP 10.20.1.109/32	Reverse ping test from App-B
ewb-sg-app-a	SSH / TCP 22 (temporary)	App-B private IP 10.20.1.109/32	Diagnostic rule for the reverse TCP test
ewb-sg-app-b (sg-08efd0ad9ba86d0e6)	SSH / TCP 22	Bastion private IP 10.10.1.211/32	Administrative SSH from the bastion
ewb-sg-app-b	All ICMP, IPv4	App-A private IP 10.10.2.59/32	Forward ping test from App-A

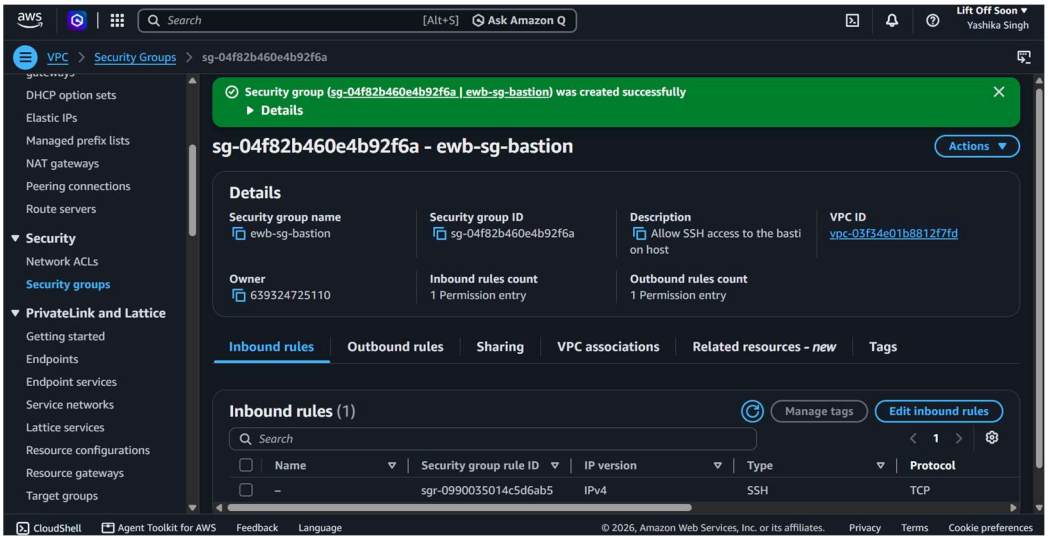


Figure 12: ewb-sg-bastion (sg-04f82b460e4b92f6a) created with one inbound SSH (TCP 22) rule for the operator address.

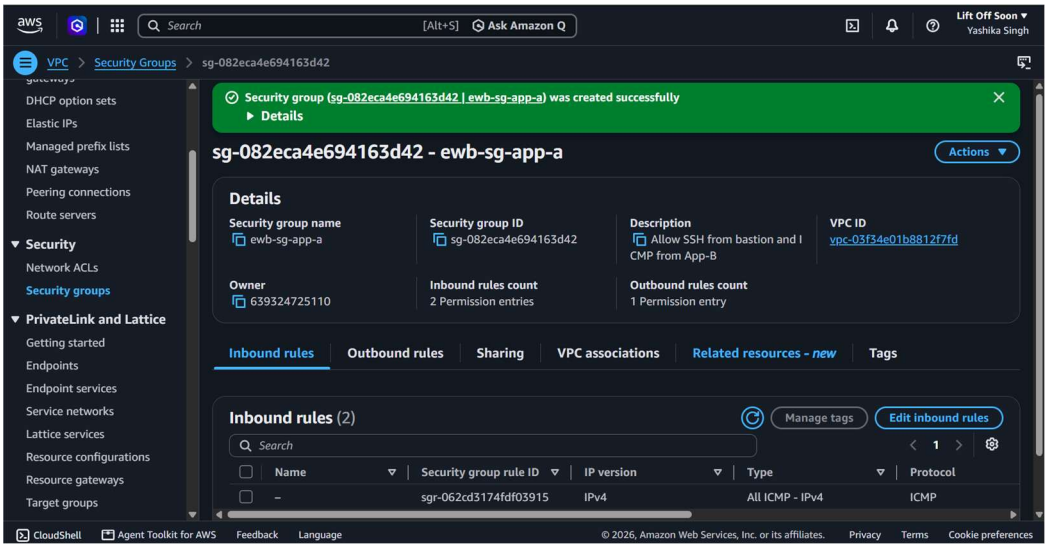


Figure 13: ewb-sg-app-a (sg-082eca4e694163d42) created with two inbound rules: an ICMP rule and an SSH rule.

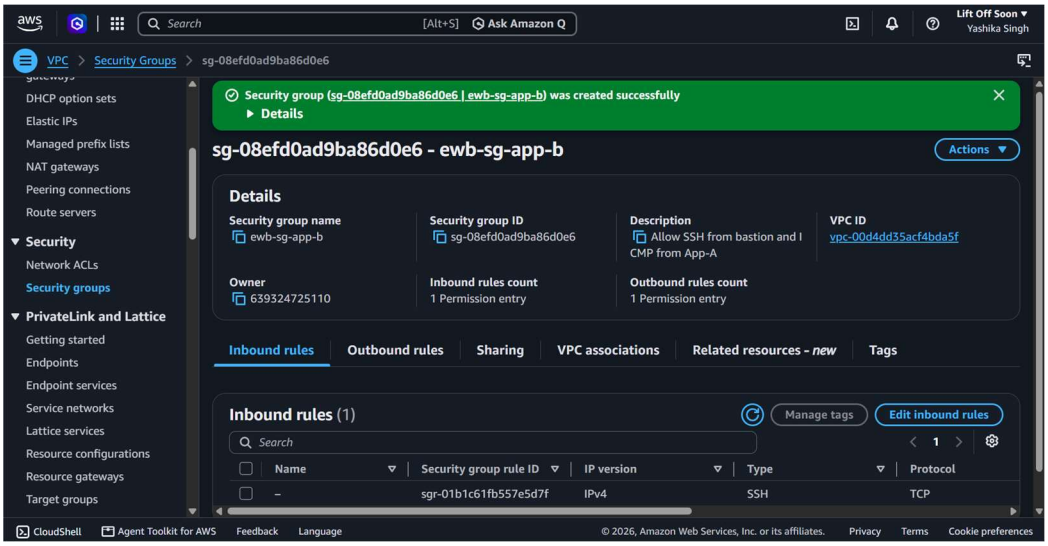


Figure 14: ewb-sg-app-b (sg-08efd0ad9ba86d0e6) created. The captured view lists one inbound rule (SSH); see Section 6.5 on the ICMP rule.

6.4 Network ACLs

The network ACLs for the application subnets were inspected. The observed inbound rules were rule 100, allowing all traffic from 0.0.0.0/0, and the default deny rule. Both network ACLs were left at the workbook's default allow-all configuration. Because network ACLs are stateless, return traffic is permitted by the default outbound rule.

6.5 Address Correction for Security-Group Rules

The security-group rules were first entered with the workbook's example addresses, 10.10.1.10 for the bastion and 10.20.1.10 for App-B, as shown in Figure 15. The instances received AWS-assigned private addresses (10.10.1.211 for the bastion, 10.20.1.109 for App-B and 10.10.2.59 for App-A), so the rules did not match the traffic. The rules were corrected to the AWS-assigned addresses. The captured SSH sessions from the bastion and the successful ping results in Section 7 were obtained with these corrected sources.

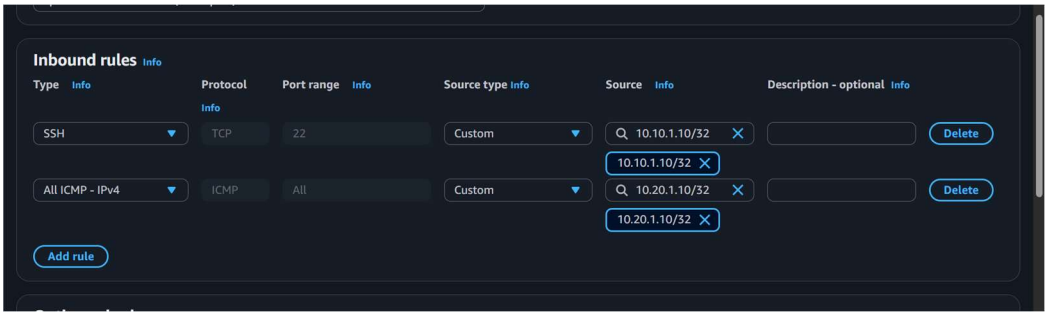


Figure 15: Inbound rules as entered for an application security group: SSH source 10.10.1.10/32 and ICMP source 10.20.1.10/32, which are the workbook's example addresses (see Section 6.5).

The workbook states that when AWS-assigned addresses are used, every instance IP and /32 rule must be replaced with the actual values. This correction is the main configuration change made during the lab.

7. Connectivity Testing

Each test was run from the named source to the named destination using the command shown. The results are those captured in the terminal screenshots. Where the captured output is partial or a result was not captured, the table says so.

Table 8: Connectivity test results

#	Test	Source to destination	Command	Captured result	Evidence
T1	Bastion reaches App-A (ICMP)	10.10.1.211 to 10.10.2.59	ping -c 4	4 sent, 4 received, 0% loss	Figure 16
T2	Bastion reaches App-B across peering (ICMP)	10.10.1.211 to 10.20.1.109	ping -c 4	4 sent, 4 received, 0% loss	Figure 17
T3	App-A reaches App-B across peering (ICMP)	10.10.2.59 to 10.20.1.109	ping -c 4	4 sent, 4 received, 0% loss	Figure 18
T4	App-A reaches App-B on TCP 22	10.10.2.59 to 10.20.1.109	bash /dev/tcp, 6 s timeout	First attempt failed; subsequent attempt succeeded	Figure 19
T5	App-A SSH to App-B	10.10.2.59 to 10.20.1.109	ssh	Service reached; refused at public-key authentication	Figure 19 (top line)
T6	App-B reaches App-A on TCP 22	10.20.1.109 to 10.10.2.59	bash /dev/tcp, 6 s timeout	Two failures, then success after the rule change	Figure 21
S1	Laptop SSH to bastion	Laptop to 3.27.124.150	ssh (ProxyCommand not used)	Login succeeded; shell on ip-10-10-1-211	Figure 8
S2	Laptop SSH to App-A and App-B through bastion	Laptop to 10.10.2.59 and 10.20.1.109	ssh -o ProxyCommand=...	Login succeeded to both hosts	Figure 20; Figure 21 (App-B shell)

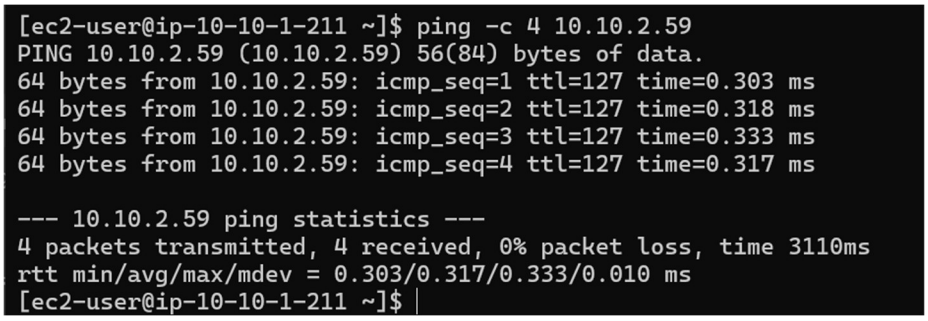


Figure 16: Bastion (10.10.1.211) pinging App-A (10.10.2.59): 4 transmitted, 4 received, 0% packet loss.

```
[ec2-user@ip-10-10-1-211 ~]$ ping -c 4 10.20.1.109
PING 10.20.1.109 (10.20.1.109) 56(84) bytes of data.
64 bytes from 10.20.1.109: icmp_seq=1 ttl=127 time=0.163 ms
64 bytes from 10.20.1.109: icmp_seq=2 ttl=127 time=0.173 ms
64 bytes from 10.20.1.109: icmp_seq=3 ttl=127 time=0.184 ms
64 bytes from 10.20.1.109: icmp_seq=4 ttl=127 time=0.179 ms

--- 10.20.1.109 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3087ms
rtt min/avg/max/mdev = 0.163/0.174/0.184/0.007 ms
[ec2-user@ip-10-10-1-211 ~]$
```

Figure 17: Bastion (10.10.1.211) pinging App-B (10.20.1.109): 4 transmitted, 4 received, 0% packet loss.

```
[ec2-user@ip-10-10-2-59 ~]$ ping -c 4 10.20.1.109
PING 10.20.1.109 (10.20.1.109) 56(84) bytes of data.
64 bytes from 10.20.1.109: icmp_seq=1 ttl=127 time=0.222 ms
64 bytes from 10.20.1.109: icmp_seq=2 ttl=127 time=0.176 ms
64 bytes from 10.20.1.109: icmp_seq=3 ttl=127 time=0.159 ms
64 bytes from 10.20.1.109: icmp_seq=4 ttl=127 time=0.185 ms

--- 10.20.1.109 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3116ms
rtt min/avg/max/mdev = 0.159/0.185/0.222/0.023 ms
[ec2-user@ip-10-10-2-59 ~]$
```

Figure 18: App-A (10.10.2.59) pinging App-B (10.20.1.109) across the peering connection: 4 transmitted, 4 received, 0% packet loss.

```
[ec2-user@ip-10-10-2-59 ~]$ timeout 6 bash -c '</dev/tcp/10.20.1.109/22' && echo "TCP CONNECTIVITY SUCCESSFUL" || echo "
TCP CONNECTION FAILED"
TCP CONNECTIVITY SUCCESSFUL
```

Figure 19: TCP port 22 test from App-A to App-B using /dev/tcp. The captured output shows a failed attempt followed by a successful one. The top line is the SSH authentication refusal from the earlier attempt (partly visible).

```
PS C:\Users\Lenovo> ssh -i "$HOME\Downloads\ewb-lab.pem" -o "ProxyCommand=ssh -i $HOME\Downloads\ewb-lab.pem -W %h:%p ec
2-user@3.27.124.150" ec2-user@10.10.2.59
The authenticity of host '10.10.2.59 (<no hostip for proxy command>)' can't be established.
ED25519 key fingerprint is SHA256:3iCaCKwX96Zu2oK4R/RGdeVeak1h9gO1Yi7Hx/8jaNg.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.2.59' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-2-59 ~]$
```

Figure 20: SSH from the laptop to App-A (10.10.2.59) through the bastion using ProxyCommand. The login succeeds and the shell is on ip-10-10-2-59.

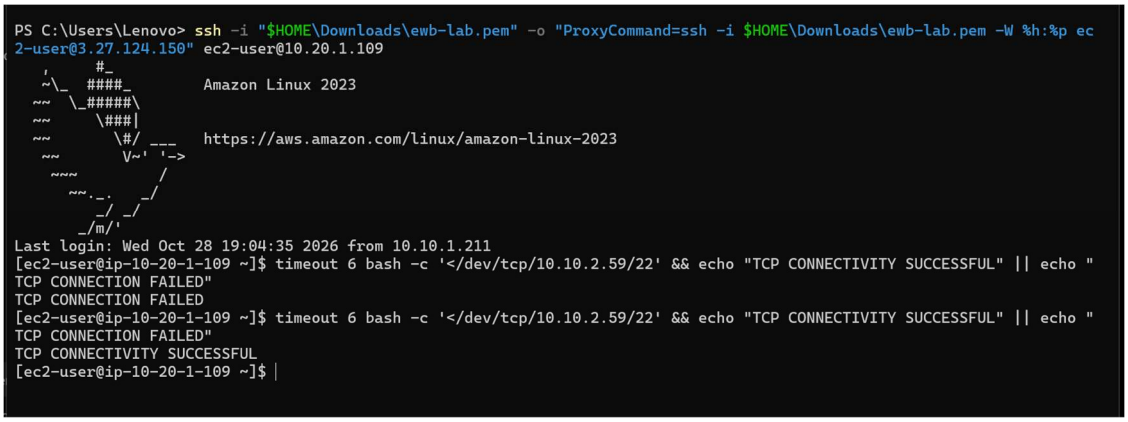


Figure 21: From an SSH session on App-B (10.20.1.109), the TCP port 22 test to App-A (10.10.2.59): two failed attempts, then success after the App-A security-group rule was added.

7.1 Test Procedure

ICMP tests used ping with four echo requests. TCP tests used the bash /dev/tcp mechanism with a six-second timeout, which reports whether a TCP handshake to a port completes. The SSH tests used the standard client, so they exercised the service and its authentication step.

7.2 Interpretation of the Results

Routing and reachability. The successful ICMP tests in both directions from the bastion, and from App-A to App-B, show that the peering connection was active and that the routes for those flows were in place. The TCP 22 test from App-B to App-A succeeded after a rule change, which shows that the return path existed and that the destination rule was the blocking factor at that point.

TCP reachability. The first App-A to App-B TCP 22 attempt in the captured output failed, and a later attempt succeeded. The cause of the first failure is not recorded, so this result is reported as observed rather than explained.

Network connectivity and SSH authentication. The T5 result shows that the SSH service on App-B responded. The authentication refusal shows that App-A did not hold a key that App-B accepts. It does not show that the network connection was broken. The private key was not copied to App-A to overcome this, which is consistent with the intended design.

Not captured. The reverse ping (App-B to App-A) was not captured. This gap is listed in Section 11.

8. Troubleshooting

The troubleshooting record covers four issues. For each, the observed facts are stated first and the possible causes are listed separately. Possible causes have not been confirmed.

8.1 Address Correction for Security-Group Rules

The security-group rules were first entered with the workbook's example addresses (Figure 15). Because the instances received AWS-assigned addresses, those rules did not match the traffic and blocked it.

Setting the sources to the AWS-assigned addresses (10.10.1.211, 10.20.1.109 and 10.10.2.59) produced the captured ping and SSH results in Section 7. The project notes also record an earlier App-A to App-B ICMP failure. That failure is consistent with the same address mismatch, but no capture of it was supplied, so it is not relied on in this report.

8.2 Reverse TCP Failure Fixed by a Security-Group Change

Observed: the test from App-B to App-A on TCP port 22 failed twice (Figure 21). After an SSH inbound rule for 10.20.1.109/32 was added to ewb-sg-app-a, the test succeeded.

Interpretation: the route back to VPC A (10.10.0.0/16 to the peering connection in ewb-rt-b-private) was in place throughout, so the failure was caused by the destination's security group rather than by routing. Adding the missing rule corrected the result. The rule was a diagnostic measure and should be reviewed (Section 10).

8.3 SSH Authentication Refused from App-A to App-B

Observed: the SSH client on App-A reached App-B's SSH service and was refused at authentication, with the message Permission denied (publickey, gssapi-keyex, gssapi-with-mic). The top line of Figure 19 shows this message.

Interpretation: the network path and the SSH service worked. The failure occurred because App-A did not hold a key App-B accepts. This matches the design intent that the private key stays on the laptop. Administrative access to App-B therefore runs through the bastion from the laptop, which was captured working (Figure 21). Agent forwarding or a second key pair would be needed for instance-to-instance SSH; neither was configured.

8.4 First TCP Attempt from App-A to App-B

The first attempt in Figure 19 failed and a later attempt succeeded. The project record does not state what changed between them, so the cause is unknown. Recording the reason would turn this into a documented troubleshooting step.

9. Results and Discussion

Table 9: Requirements against results

Requirement	Result	Evidence
Non-overlapping CIDRs and peering	Achieved: 10.10.0.0/16 and 10.20.0.0/16; pcx-095011188f7b136a1 Active	Figure 2; Figure 3; Figure 4
Application instances remain private	Achieved: no public IPv4 on App-A or App-B; no default route in their tables; VPC B has no Internet Gateway	Figure 5; Figure 10; Figure 11
Forward and return routes	Achieved: 10.20.0.0/16 to pcx in VPC A tables; 10.10.0.0/16 to pcx in ewb-rt-b-private	Figure 9 to Figure 11

Requirement	Result	Evidence
Bastion reaches both application hosts	Achieved: ICMP 4 of 4 to each	Figure 16; Figure 17
Private App-A to App-B connectivity	Achieved for ICMP (4 of 4) and for TCP 22 after a first failed attempt	Figure 18; Figure 19
Reverse TCP (App-B to App-A)	Achieved after one security-group change: two failures, then success	Figure 21
Administration without copying keys	Achieved: SSH to App-A and App-B through the bastion using ProxyCommand	Figure 20; Figure 21
Instance-to-instance SSH	Not achieved: authentication refused (no key on App-A)	Figure 19
Least-privilege inbound rules	Achieved: application groups admit SSH only from the bastion and ICMP only from the peer, by /32 source	Figure 12 to Figure 14

The project established the core VPC peering infrastructure: two non-overlapping VPCs, an active peering connection, routes in both directions, private instances without public addresses, and administrative access through a single bastion. Connectivity between the application instances was shown by ICMP in the direction tested, and by TCP 22 once a rule was corrected. Instance-to-instance SSH was not configured.

The main lesson is that an Active peering connection alone does not make traffic flow. Routes on both sides, a permissive rule at the destination, and the correct source address in that rule were each needed. The reverse TCP result is a clear example: the route was present, and a missing security-group rule caused the failure.

10. Security Considerations

Private addressing. Application instances have no public IPv4 address and no internet route. VPC B has no Internet Gateway, and peering does not provide one.

Bastion as single entry point. The bastion is the only resource with a public IPv4 address. Its SSH rule is restricted to the operator's /32 address.

Least-privilege intent. Application security groups accept traffic only from specific /32 sources, and ICMP is limited to the peer instance. No rule uses 0.0.0.0/0. The temporary diagnostic rule is described below.

Diagnostic rules. The temporary SSH rule on ewb-sg-app-a (from 10.20.1.109/32) was added for testing. It should be removed or justified before the environment is used again.

ICMP scope. The ICMP rules use All ICMP IPv4 rather than echo request only. Narrowing them to echo request would tighten the policy.

Key handling. The private key ewb-lab.pem was kept on the local computer and never copied to an EC2 instance. ProxyCommand keeps the key on the laptop during both hops. The key pair and local file should be removed once the lab is complete, if no longer needed.

Host key verification. The first connection to each host should be verified before accepting its host key. The bastion connection in the captures prompted for this and was accepted after the fingerprint was shown.

Bastion exposure. The public bastion should be terminated promptly after testing. If the operator's IP changes, its security-group rule must be updated.

11. Limitations and Future Improvements

11.1 Limitations

- The reverse ping (App-B to App-A) was not captured.
- The first App-A to App-B TCP 22 attempt failed and its cause is not recorded.
- Instance-to-instance SSH was not configured, so it is not demonstrated.
- The outbound security-group rules and outbound network ACL rules are not part of the captured evidence.
- No cleanup evidence (terminated instances, deleted peering, subnets, route tables, gateway, VPCs, or key pair) was among the screenshots supplied, so cleanup is not claimed.
- All testing used one account, one Region, and one Availability Zone; results do not describe cross-account, cross-Region, or cross-AZ peering.
- Only ICMP and TCP port 22 were tested. No application-layer protocol was tested.

11.2 Future Improvements

- Capture the reverse ping (App-B to App-A) and the outbound rule configuration.
- Narrow the ICMP rules to echo request only, and remove the diagnostic SSH rule.
- Configure SSH agent forwarding or a separate key pair, so that instance-to-instance SSH can be tested without copying the private key.
- Enable VPC Flow Logs to show accepted and rejected traffic in each direction.
- Use an Elastic IP for the bastion, or document its changing address, so that the SSH configuration remains stable.

12. Conclusion

The project implemented two non-overlapping VPCs in the Sydney Region, connected them with an active VPC peering connection, and configured routes in both directions. The application instances remained free of public addresses, and administrative access went through a single bastion using SSH ProxyCommand, so no private key was stored on any server.

The captured tests show ICMP reachability from the bastion to both application instances and from App-A to App-B, and a TCP 22 connection between the application instances once the destination rule was corrected. SSH from the laptop reached both private instances through the bastion. Instance-to-instance SSH authentication was not configured, the reverse ping was not captured.

What was learned:

- An active peering connection is only a link. Routes on both sides and a permissive rule at the destination determine whether traffic flows.
- Routes are one-way. The return route is as necessary as the forward route.
- With ProxyCommand, the target sees the bastion's private IP as the source, so the SSH rule must reference the bastion's address.
- Security-group rules must use the addresses AWS actually assigns. Placeholder addresses from a workbook silently block traffic.
- Peering is non-transitive and gives no edge-to-edge routing. VPC B cannot use VPC A's Internet Gateway.

References

[1] EWB Courses, AWS VPC Peering: Student Project Workbook (Project 01, AWS Architect Associate), October 2026.

[2] Amazon Web Services, Amazon VPC User Guide: VPC peering concepts and limitations (docs.aws.amazon.com/vpc).

[3] Amazon Web Services, Amazon VPC User Guide: Create a VPC peering connection, and Update route tables for peering (docs.aws.amazon.com/vpc).

[4] Amazon Web Services, Amazon VPC User Guide: Security groups and network ACLs (docs.aws.amazon.com/vpc).

[5] Amazon Web Services, Amazon EC2 User Guide: Amazon EC2 key pairs (docs.aws.amazon.com/ec2).

[6] OpenSSH manual pages: ssh_config(5) (ProxyJump and ProxyCommand) and ssh(1).

Appendix A. Figure Index

Each figure is a screenshot supplied by the student (the arch figure is a drawing for this report). Redaction status is stated for each figure.

Table 10: Figure index

Figure	Content	Status	Redaction
1	Network architecture of the project	Included	Not redacted
2	Peering connection ewb-a-b-peer (pcx-095011188f7b136a1) in Active status	Included	Not redacted
3	VPC A, ewb-vpc-a (vpc-03f34e01b8812f7fd), created as VPC only with IPv4 CIDR 10	Included	Not redacted
4	VPC B, ewb-vpc-b (vpc-00d4dd35acf4bda5f), IPv4 CIDR 10	Included	Not redacted
5	EC2 instances ewb-bastion, ewb-app-a and ewb-app-b on t3	Included	Not redacted
6	Internet gateway igw-079fd165b4c4ccae7 (ewb-igw-a) attached to VPC A (vpc-03f34e01b8812f7fd)	Included	Not redacted
7	Creation of the RSA key pair ewb-lab, downloaded in	Included	Not redacted
8	First SSH session from the laptop to ewb-bastion (3	Included	Not redacted
9	ewb-rt-a-public (rtb-069a40c949c665db4) after the route update: 0	Included	Not redacted
10	ewb-rt-a-private (rtb-0f5d0567dfce2bee9): local route 10	Included	Not redacted
11	ewb-rt-b-private (rtb-0e05059307348f9db): return route 10	Included	Not redacted
12	ewb-sg-bastion (sg-04f82b460e4b92f6a) created with one inbound SSH (TCP 22) rule for the operator address	Included	Not redacted
13	ewb-sg-app-a (sg-082eca4e694163d42) created with two inbound rules: an ICMP rule and an SSH rule	Included	Not redacted
14	ewb-sg-app-b (sg-08efd0ad9ba86d0e6) created	Included	Not redacted
15	Inbound rules as entered for an application security group: SSH source 10	Included	Not redacted
16	Bastion (10	Included	Not redacted
17	Bastion (10	Included	Not redacted
18	App-A (10	Included	Not redacted
19	TCP port 22 test from App-A to App-B using /dev/tcp	Included	Not redacted
20	SSH from the laptop to App-A (10	Included	Not redacted
21	From an SSH session on App-B (10	Included	Not redacted

Account IDs and the console header name appear in several screenshots and have not been redacted.

Appendix B. Evidence Coverage

This table lists what the captured screenshots establish and what they do not.

Table 11: Evidence coverage

Item	Status	Figure
Peering connection Active	Captured	Figure 2
VPC IDs and CIDRs	Captured	Figure 3; Figure 4
Internet gateway attached to VPC A	Captured	Figure 6
Three route tables and their routes	Captured	Figure 9 to Figure 11
Key pair ewb-lab created	Captured	Figure 7
Security groups and inbound rules	Captured	Figure 12 to Figure 14
SSH to bastion, App-A and App-B	Captured	Figure 8; Figure 20; Figure 21
Ping: bastion to App-A and App-B	Captured	Figure 16; Figure 17
Ping: App-A to App-B	Captured	Figure 18
TCP 22: App-B to App-A (after rule change)	Captured	Figure 21
TCP 22: App-A to App-B	Captured (first attempt failed)	Figure 19
SSH App-A to App-B authentication	Captured (top line only)	Figure 19
Ping: App-B to App-A	Not captured	None
Outbound security-group and NACL rules	Not captured	None
Cleanup of lab resources	Not captured	None